PROJECT REPORT ON

HOSTEL MANAGEMENT SYSTEM USING PYTHON, TKINTER AND MYSQL

(Submitted in partial fulfilment of the requirement for the
Python Development Training Course conducted at ZillionSoftech, Rewari)

By

YUVRAJ SINGH TANWAR

ZillionSoftech
Rewari, Haryana

Session: 2025–26

Year of Submission: 2026

CERTIFICATE

Certified that this project report entitled “Hostel Management System using Python, Tkinter and MySQL” is an original piece of work carried out by Yuvraj Singh Tanwar during the Python Development Training Course undertaken at ZillionSoftech, Rewari, under the guidance of the Course Trainer/Mentor, ZillionSoftech, Rewari.

Dated: _____

Signature of Trainee: _____

Name: Yuvraj Singh Tanwar

Institute: ZillionSoftech, Rewari

Countersigned by:

_____

Course Trainer / Mentor

ZillionSoftech, Rewari

ACKNOWLEDGMENTS

The successful completion of this project would not have been possible without the guidance, support, and encouragement of many people, and I take this opportunity to express my sincere gratitude to all of them.

I am deeply thankful to the Director, ZillionSoftech, Rewari, for providing me the opportunity and a well-equipped learning environment to undertake this Python Development Training Course.

I express my heartfelt gratitude to my Course Trainer/Mentor at ZillionSoftech, Rewari, whose constant guidance, technical expertise, and patient mentoring helped me understand real-world software development practices in Python and complete my assigned project successfully.

I would also like to thank all the staff members of ZillionSoftech, Rewari, and my fellow trainees who directly or indirectly supported me during the course of this training.

Finally, I am grateful to my family and friends for their constant motivation and support.

Yuvraj Singh Tanwar

TABLE OF CONTENTS

1. ABOUT THE INSTITUTE

ZillionSoftech is a technical training and software development institute located at Rewari, Haryana. The institute works in the areas of software development, desktop and web application development, data analytics, and IT skill training. It conducts industry-oriented, project-based training programmes in which trainees work on live, real-world projects under the mentorship of experienced professionals.

1.1 Vision and Mission

The vision of ZillionSoftech is to bridge the gap between academic learning and industry requirements by producing skilled, employable, and industry-ready professionals. Its mission is to impart practical, hands-on training on widely used technologies such as Python, MySQL, Java, and data analytics tools, and to deliver quality software solutions to its clients.

1.2 Areas of Work

- Application development using Python (Desktop GUI applications using Tkinter, automation scripts, and data processing)
- Database design and management using MySQL
- Data Analytics using Python (Pandas, NumPy), Excel, and Power BI
- Web Development and Digital Marketing
- Corporate training, student courses, and placement assistance programmes

1.3 Organizational Structure

The institute functions under a Director, supported by project managers, senior developers, and trainers. Trainees are attached to a project batch and work under the direct guidance of a senior developer/trainer who acts as the Course Mentor. Weekly review sessions are conducted to monitor the progress of the trainees and the project deliverables.

1.4 Work Culture and Facilities

The work culture of ZillionSoftech is collaborative and learning-oriented. Daily task allotment, regular code reviews, task tracking, and proper documentation are integral parts of the workflow. The institute provides a fully equipped computer lab with the required software (Python, MySQL Server, MySQL Workbench, VS Code/PyCharm) and internet facility, which helped me understand the professional practices followed in the software industry.

1.5 Why This Institute Was Chosen

I chose ZillionSoftech for my training because it offers project-based, hands-on training on Python and MySQL, which are directly related to my area of interest and career goals. Being conveniently located in Rewari, it also allowed me to attend the training regularly and complete a complete real-world project under proper mentorship.

2. OBJECTIVES OF THE COURSE

This training was undertaken as a project-based Python Development Course at ZillionSoftech, Rewari, aimed at building industry-ready practical skills in desktop application development. The main objectives of this course were:

2.1 General Objectives

- To gain first-hand practical experience of working in a professional software development environment and to understand the working culture of an IT organization.
- To bridge classroom/theoretical learning with real, hands-on project work.
- To develop professional competencies such as teamwork, communication, time management, problem-solving, discipline, and dedication.
- To enhance employability skills and build confidence for future job roles in the field of software development.

2.2 Technical Objectives

- To learn desktop application development in Python using the Tkinter GUI toolkit.
- To design and implement a relational database in MySQL and connect it with a Python application using the mysql-connector-python library.
- To understand and implement the MVC (Model-View-Controller) software architecture in a real project.
- To implement complete CRUD (Create, Read, Update, Delete) operations, user authentication, input validation, and report generation.
- To understand the complete Software Development Life Cycle (SDLC) including requirement analysis, design, development, testing, and documentation.
- To learn the use of industry-standard tools such as Git/GitHub for version control, VS Code/PyCharm for development, and MySQL Workbench for database management.

2.3 Expected Outcome

At the end of the course, the expected outcome was a fully working desktop-based Hostel Management System developed in Python with a Tkinter GUI and a MySQL backend, following the MVC architecture, along with complete project documentation, which could be used by a hostel/institute to manage its rooms, residents, wardens, and fee collection.

3. DETAILS OF WORK DONE

3.1 Project Overview

During the course, I worked on a live project titled "Hostel Management System" — a desktop-based application developed using Python, Tkinter, and MySQL. The objective of the project was to computerize the day-to-day activities of a hostel, such as room allotment, resident (student) admission, warden/staff management, fee collection, and complaint handling, which are otherwise maintained manually in registers.

The application follows the MVC (Model-View-Controller) architecture, in which the GUI (Views), the business/application logic (Controllers), and the data and database operations (Models) are kept in separate layers. This separation of concerns makes the application easy to understand, test, maintain, and extend.

Main Features of the Application

- Secure login system with hashed passwords and role-based access (Admin / Warden).
- Resident management: add, update, delete, search, and view hostel residents in a tabular form.
- Room management: add, update rooms, allot/vacate rooms, and track occupancy and availability.
- Warden/Staff management: add, update warden/staff records and assign them to hostel blocks.
- Fee management: collect hostel fee, view complete fee history, and automatically calculate pending fee of each resident.
- Complaint management: residents' complaints (maintenance, mess, discipline, etc.) can be logged, tracked, and marked resolved.
- Dashboard displaying total residents, rooms, occupancy, and total fee collected.
- Report generation and export of resident and fee reports to PDF, Excel, and CSV formats.
- Application logging (logs/application.log) and database backup facility.

3.2 Technologies and Tools Used

Category	Technology / Tool	Purpose in the Project
Programming Language	Python 3.12	Core application logic, business rules, and all backend processing
GUI Toolkit	Tkinter (with ttk widgets)	Designing all windows, forms, tables, buttons, and dialog boxes of the desktop application
Database	MySQL 8.0	Permanent storage of residents, rooms, wardens, fees, and user records
DB Connector	mysql-connector-python	Connecting the Python application to the MySQL database and executing SQL queries
Database Tool	MySQL Workbench	Creating and testing the database

Category	Technology / Tool	Purpose in the Project
		schema, running SQL scripts, and viewing table data
IDE / Editor	VS Code / PyCharm	Writing, debugging, and organizing the project code
Version Control	Git and GitHub	Maintaining the project repository, tracking changes, and commit history
Reporting Libraries	reportlab, openpyxl, csv	Exporting resident and fee reports to PDF, Excel, and CSV formats
Other Libraries	hashlib, logging, datetime	Password hashing for login security, application logging, and date handling

3.3 System Architecture

The project follows the MVC (Model-View-Controller) architecture. Each layer has a single, well-defined responsibility, and the layers communicate with each other in a fixed direction, as described below:

- Views (Tkinter GUI): This layer contains all the screens of the application (login window, dashboard, and forms for residents, rooms, wardens, fees, and complaints). The views only collect input from the user and display output; they do not contain any database code.
- Controllers: Controllers act as a bridge between the views and the models. When the user clicks a button (e.g., “Save Resident”), the view calls the corresponding controller, which validates the data and then calls the appropriate model method.
- Models: This layer contains the data classes as well as all the SQL/database operations of the application. Each entity has its own model class (Resident, Room, Warden, Fee, Complaint, User) which represents a record and also performs INSERT, SELECT, UPDATE, and DELETE operations using parameterized queries to prevent SQL injection.
- Database (MySQL): The MySQL database hostel_db permanently stores all the data of the application in five related tables.

Flow of a Typical Operation (Allot Room to Resident)

User fills the Add Resident form (View) → clicks Save → resident_controller.py receives the data → data is validated → a Resident model object is created → resident_model.py executes the INSERT query through db_connection.py, and the room's occupancy count is updated → the result (success/failure) travels back to the view → a message box is shown to the user and the resident list (Treeview) is refreshed.

3.4 Project Directory Structure (MVC)

The complete project was organized in the following directory structure, which reflects the MVC architecture described above:

```
HostelManagementSystem/
|-- main.py                  # Application entry point
|-- config.py                # Configuration settings (DB credentials)
|-- requirements.txt         # Required Python libraries
```

```
|-- README.md                # Project documentation
|
|-- database/                # Database connection & creation scripts
|   |-- db_connection.py     # MySQL database connection
|   |-- create_database.py   # Creates database & tables
|   |-- backup.py            # Database backup utility
|   |-- sql/hostel_db.sql    # SQL script of the schema
|
|-- models/                  # Model layer (data + database operations)
|   |-- resident_model.py    # Resident data class + DB operations
|   |-- room_model.py        # Room data class + DB operations
|   |-- warden_model.py      # Warden/Staff data class + DB operations
|   |-- fee_model.py         # Fee data class + DB operations
|   |-- complaint_model.py   # Complaint data class + DB operations
|   |-- user_model.py        # User/login data class + DB operations
|
|-- controllers/             # Controller layer (connects views with models)
|   |-- login_controller.py, resident_controller.py,
|   |-- room_controller.py, warden_controller.py,
|   |-- fee_controller.py, complaint_controller.py, dashboard_controller.py
|
|-- views/                   # Tkinter GUI screens
|   |-- login.py, dashboard.py
|   |-- resident/  (add, update, delete, search, list)
|   |-- room/      (add, update, allot, vacate, list)
|   |-- warden/    (add, update, list)
|   |-- fee/       (collect_fee, fee_history, pending_fee)
|   |-- complaint/ (raise_complaint, complaint_list)
|   |-- reports/   (resident, fee, occupancy reports)
|
|-- utils/                   # Helper utilities & validation
|   |-- constants.py, helper.py, validator.py,
|   |-- date_util.py, messagebox_util.py
|   |-- export_util.py       # Report export to PDF/Excel/CSV
|
|-- assets/                  # Images, icons, logo
|-- reports/                 # Generated PDF / Excel / CSV reports
|-- logs/                    # application.log
|-- tests/                   # Unit tests (resident, room, login)
```

3.5 Database Design

A relational database named hostel_db was designed in MySQL after analysing the requirements of the hostel. The schema was normalized up to the Third Normal Form (3NF) to avoid data redundancy. The database consists of the following five tables:

3.5.1 Tables in the Database

Sr. No.	Table Name	Description
1	users	Stores login credentials (username, hashed password, role) of admin and warden users
2	residents	Stores personal and academic/guardian details of every student residing in the hostel
3	rooms	Stores details of hostel rooms, their type, capacity, and current occupancy
4	wardens	Stores details of wardens/staff and the block/floor assigned to

Sr. No.	Table Name	Description
		them
5	fees	Stores hostel fee payment transactions of the residents along with pending amount details

3.5.2 Structure of the residents Table

Field Name	Data Type	Description / Constraint
resident_id	INT	Primary Key, Auto Increment
name	VARCHAR(100)	Full name of the resident, NOT NULL
guardian_name	VARCHAR(100)	Parent/guardian name of the resident
dob	DATE	Date of birth
gender	ENUM('M','F','O')	Gender of the resident
mobile	VARCHAR(10)	10-digit mobile number, UNIQUE
email	VARCHAR(100)	Email address of the resident
address	VARCHAR(255)	Permanent/home address
room_id	INT	Foreign Key referencing rooms(room_id)
admission_date	DATE	Date of admission to the hostel, default current date
status	ENUM('Active','Left')	Current status of the resident

3.5.3 Structure of the rooms Table

Field Name	Data Type	Description / Constraint
room_id	INT	Primary Key, Auto Increment
room_no	VARCHAR(10)	Room number, UNIQUE
room_type	ENUM('Single','Double','Triple')	Type/category of the room
capacity	INT	Maximum number of residents allowed
occupied	INT	Current number of residents occupying the room
floor	VARCHAR(20)	Floor/block on which the room is located
status	ENUM('Available','Full')	Current occupancy status of the room, derived from occupied vs capacity

3.5.4 Structure of the fees Table

Field Name	Data Type	Description / Constraint
fee_id	INT	Primary Key, Auto Increment

Field Name	Data Type	Description / Constraint
resident_id	INT	Foreign Key referencing residents(resident_id)
amount_paid	DECIMAL(10,2)	Amount paid in the current transaction
payment_date	DATE	Date of the fee payment
payment_mode	ENUM('Cash','UPI','Card')	Mode of the payment
remarks	VARCHAR(255)	Optional remarks such as instalment number

3.5.5 Relationships between Tables

- rooms (1) —— (many) residents: one room can accommodate many residents (residents.room_id is a foreign key).
- residents (1) —— (many) fees: one resident can make many fee payments (fees.resident_id is a foreign key).
- wardens (1) —— (many) rooms: wardens are mapped to the blocks/floors they supervise.

The pending fee of a resident is calculated dynamically using a JOIN query as: Pending Fee = hostel_fee_structure.total_fee – SUM(fees.amount_paid), so that the pending amount is always accurate and never stored redundantly.

3.6 Description of Modules

3.6.1 Login Module

The login module is the entry point of the application (views/login.py). The user enters a username and password, which are verified by the User model against the users table. Passwords are never stored in plain text; they are hashed using the SHA-256 algorithm (hashlib) before storing and comparing. On successful login, the dashboard opens according to the role of the user; on failure, an error message box is displayed and the attempt is logged.

3.6.2 Dashboard Module

The dashboard (views/dashboard.py) is the main window of the application. It displays summary cards showing the total number of active residents, total rooms, occupied/available rooms, and the total fee collected in the current month. It also contains a menu/side panel through which the user can navigate to all the other modules of the application.

3.6.3 Resident Module

This is the core module of the project. It provides five screens — Add Resident, Update Resident, Delete Resident, Search Resident, and Resident List. The resident list is displayed in a ttk.Treeview widget with scrollbars, and a resident can be searched by name, mobile number, or room number. All inputs (mobile number, email, date of birth) are validated before saving.

3.6.4 Room Module

The room module manages all hostel rooms — adding new rooms, updating room type/capacity, allotting a room to a resident, and vacating a room. The occupancy count of a room is automatically updated whenever a resident is allotted to or removed from it, and a room that is already full cannot be allotted further, which maintains data consistency.

3.6.5 Warden/Staff Module

The warden module stores the details of wardens/staff along with their qualification and the block/floor assigned to them. The Add Warden and Update Warden screens work in the same pattern as the resident module.

3.6.6 Fee Module

The fee module contains three screens: Collect Fee (records a new hostel fee payment with amount, date, and mode), Fee History (shows all payments of a selected resident), and Pending Fee (lists all residents whose paid amount is less than the applicable hostel fee, using a JOIN and GROUP BY query). A fee receipt can be printed/exported after each collection.

3.6.7 Complaint Module

The complaint module allows a resident's complaint (maintenance, mess, discipline, etc.) to be logged with a type, description, and date, and allows the warden/admin to update its status to Resolved after action is taken. This helps the hostel administration track and close pending issues systematically.

3.6.8 Reports Module

The reports module generates the Resident Report, Fee Report, and Room Occupancy Report. Using the export utility (utils/export_util.py), every report can be exported to PDF (reportlab), Excel (openpyxl), or CSV format and is saved in the reports/ directory with a date-stamped file name.

3.6.9 Utilities and Logging

The utils/ package contains common helpers — constants, validators (mobile, email, date), date utilities, and a message box wrapper for showing uniform success/error/confirmation dialogs. All important events and errors of the application are written to logs/application.log using Python's logging module, which was very useful during debugging.

3.7 Sample Code Snippets

Snippet 1: MySQL Database Connection (database/db_connection.py)

```
import mysql.connector
from mysql.connector import Error
import config

def get_connection():
    """Returns a connection object to the hostel_db database."""
    try:
        conn = mysql.connector.connect(
            host=config.DB_HOST,
            user=config.DB_USER,
            password=config.DB_PASSWORD,
            database=config.DB_NAME
        )
        return conn
    except Error as e:
        print(f"Database connection failed: {e}")
        return None
```

Snippet 2: Insert Operation in the Model Layer (models/resident_model.py)

```
from database.db_connection import get_connection
```

```
class ResidentModel:

    def add_resident(self, r):
        """Inserts a new resident record using a parameterized query."""
        query = ("INSERT INTO residents "
                 "(name, guardian_name, dob, gender, mobile, email, "
                 "address, room_id, admission_date) "
                 "VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)")
        values = (r.name, r.guardian_name, r.dob, r.gender,
                  r.mobile, r.email, r.address,
                  r.room_id, r.admission_date)
        conn = get_connection()
        try:
            cursor = conn.cursor()
            cursor.execute(query, values)
            conn.commit()
            return cursor.lastrowid
        finally:
            conn.close()
```

Snippet 3: Add Resident Form using Tkinter (views/resident/add_resident.py)

```
import tkinter as tk
from tkinter import ttk
from controllers.resident_controller import ResidentController
from utils.messagebox_util import show_success, show_error

class AddResidentWindow(tk.Toplevel):

    def __init__(self, master):
        super().__init__(master)
        self.title("Add New Resident")
        self.geometry("450x420")
        self.controller = ResidentController()

        tk.Label(self, text="Resident Name").grid(row=0, column=0,
                 padx=10, pady=8, sticky="w")
        self.name_entry = tk.Entry(self, width=30)
        self.name_entry.grid(row=0, column=1)

        # ... similar widgets for mobile, email, room etc. ...

        tk.Button(self, text="Save", width=12,
                  command=self.save_resident).grid(row=8, column=1)

    def save_resident(self):
        result = self.controller.add_resident(
            name=self.name_entry.get(),
            mobile=self.mobile_entry.get(),
            email=self.email_entry.get(),
            room_id=self.room_combo.get()
        )
        if result["success"]:
            show_success("Resident added successfully!")
        else:
            show_error(result["message"])
```

Snippet 4: Input Validation (utils/validator.py)

```
import re
```

```
def is_valid_mobile(mobile):
    """Mobile must be exactly 10 digits starting with 6-9."""
    return bool(re.fullmatch(r"[6-9][0-9]{9}", mobile))

def is_valid_email(email):
    pattern = r"^[\w.+-]+@[\w-]+\.[\w.]+$"
    return bool(re.fullmatch(pattern, email))
```

3.8 Testing of the Application

Testing was carried out at two levels during the last phase of the course:

- Unit Testing: Unit tests were written in the tests/ package (test_resident.py, test_room.py, test_login.py) using Python's unittest framework. The model methods and validation functions were tested with valid, invalid, and boundary inputs (e.g., 9-digit and 11-digit mobile numbers, duplicate mobile numbers, empty fields, wrong passwords).
- Manual / Functional Testing: Every screen of the application was tested manually against a test-case sheet. Navigation flow, message boxes, Treeview refresh after every operation, room occupancy updates, pending fee calculation, and report exports were verified. The bugs found (e.g., crash on vacating a room with no resident, wrong pending fee for a resident with no payment) were fixed and re-tested.

After successful testing, the final build of the application was demonstrated to the Course Trainer/Mentor along with the project documentation and user manual.

3.9 Week-wise Work Schedule

Week	Module / Activity	Description of Work Done
Week 1	Orientation, Python & Tkinter Basics	Induction session and introduction to the team and project; revision of Python fundamentals (OOP, functions, exception handling); learned Tkinter widgets (Label, Entry, Button, Treeview, Frame) and geometry managers; set up the development environment (Python, MySQL, VS Code, Git).
Week 2	Database Design & Connectivity	Analysed the requirements of the Hostel Management System; designed the ER diagram and normalized database schema; created the hostel_db database and all tables using SQL scripts; implemented db_connection.py using mysql-connector-python with parameterized queries.
Week 3	Login & Resident Module	Developed the login window with password hashing (hashlib) and role-based access; developed the complete Resident module — add, update, delete, search, and list residents using Tkinter forms and Treeview; implemented model classes for all database operations.
Week 4	Room, Warden & Fee Modules	Developed the Room module with full CRUD operations, room allotment and vacating logic; developed the Warden/Staff module; developed the Fee module — fee collection form, fee history, and pending fee calculation using SQL JOIN queries; implemented input validation for all forms.
Week 5	Dashboard, Complaints,	Designed the main dashboard showing summary counts of

Week	Module / Activity	Description of Work Done
	Reports & Integration	residents, rooms, occupancy, and fees collected; developed the Complaint module; developed report generation and export to PDF (reportlab), Excel (openpyxl), and CSV; integrated all modules through controllers and tested the navigation flow.
Week 6	Testing, Documentation & Presentation	Performed unit testing (tests/) and manual testing of all modules; fixed the reported bugs; added application logging; prepared the README, user manual, and this project report; delivered the final project demonstration before the mentor and team.

3.10 Roles and Responsibilities

- Designing Tkinter forms and windows for the assigned modules as per the given layout.
- Writing model classes with parameterized SQL queries for all database operations.
- Implementing input validation and error handling in every form.
- Maintaining the project repository on GitHub with meaningful commit messages.
- Attending daily task-allotment meetings and reporting progress to the mentor.
- Writing unit tests and preparing module-wise documentation of the work completed.

3.11 Screenshots of the Application

The screenshots of the important screens of the developed application are given below:

Fig. 1: Login Window

[Paste Screenshot: Login Window]

Fig. 2: Main Dashboard

[Paste Screenshot: Main Dashboard]

Fig. 3: Add Resident Form

[Paste Screenshot: Add Resident Form]

Fig. 4: Resident List (Treeview)

[Paste Screenshot: Resident List Screen]

Fig. 5: Room Allotment Screen

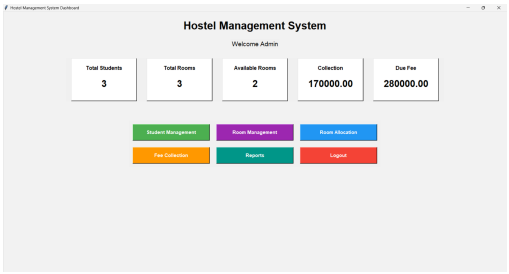
[Paste Screenshot: Room Allotment Screen]

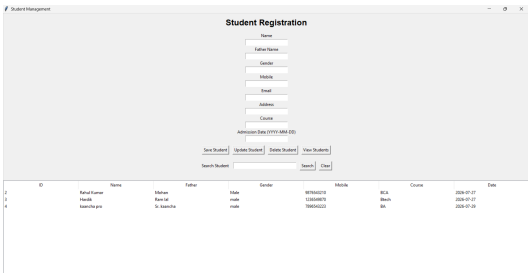
Fig. 6: Fee Collection Screen

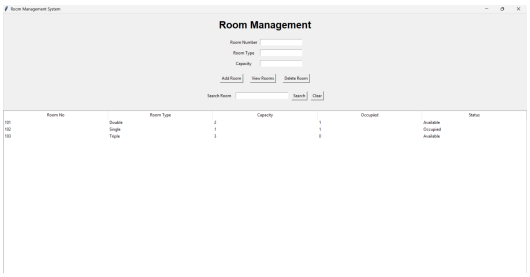
[Paste Screenshot: Fee Collection Screen]

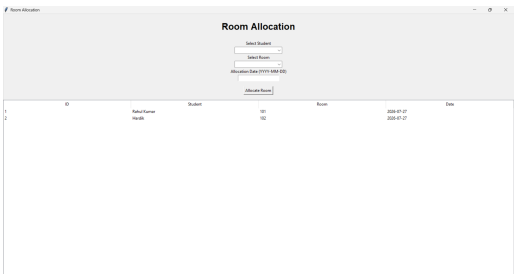
Fig. 7: Pending Fee Report / Exported PDF Report

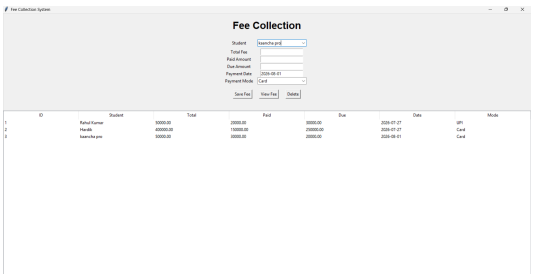
[Paste Screenshot: Pending Fee Report]











ID	Student	Total	Paid	Due	Date	Mode
1	Rahul Kumar	50000.00	20000.00	30000.00	2026-07-27	UPI
2	Hardik	400000.00	150000.00	250000.00	2026-07-27	Card
3	kaancha pro	50000.00	30000.00	20000.00	2026-08-01	Card

4. LEARNING EXPERIENCES

This course provided me with a rich and multi-dimensional learning experience that went far beyond classroom learning. Some of the key experiences are described below:

4.1 Technical Experiences

- Working on a live project helped me understand how a complete desktop application is planned, divided into layers and modules, developed, and integrated step by step, instead of writing everything in a single file as is often done in classroom practice.
- I understood the real importance of the MVC architecture — when a bug appeared in the pending fee calculation, I only had to check the model layer without touching any GUI code.
- I learned professional database practices: normalization, foreign keys, JOIN and GROUP BY queries, parameterized queries to prevent SQL injection, and taking regular database backups.
- Building GUIs in Tkinter taught me practical concepts such as geometry management (grid/pack), event-driven programming, Toplevel windows, and displaying tabular data in Treeview widgets with scrollbars.
- I experienced the practical use of version control (Git) — committing daily work, writing meaningful commit messages, and restoring a previous version when a change broke the application.
- Debugging real errors using the application log file, reading official documentation, and searching for solutions independently improved my problem-solving ability significantly.

4.2 Professional Experiences

- Following a fixed daily schedule, meeting deadlines, and reporting work honestly in daily meetings taught me discipline and accountability.
- Participating in code reviews improved my communication skills and my ability to accept and implement constructive feedback.
- Preparing the user manual and demonstrating the project to non-technical staff taught me how to explain technical concepts in simple language.
- I observed how professionalism, punctuality, teamwork, and dedication are valued in a real workplace.

4.3 Challenges Faced and How They Were Overcome

- Understanding the existing structure: In the first week, the multi-folder MVC project structure was confusing. My mentor explained the flow of one complete operation (Allot Room to Resident) across all layers, after which the entire structure became clear.
- Database connectivity errors: Initially I faced authentication and 'Access denied' errors while connecting Python to MySQL. Reading the error messages carefully and configuring the credentials in config.py resolved the issue.
- Refreshing the Treeview: After add/update/delete operations, the resident list did not refresh automatically. I learned to clear and reload the Treeview after every successful database operation.

- Pending fee logic: Residents having no fee record were not appearing in the pending list. Using a LEFT JOIN with IFNULL(SUM(amount_paid), 0) solved this problem and taught me an important SQL concept.

5. LEARNING OUTCOMES

On successful completion of this course, the following learning outcomes were achieved:

- Ability to design and develop a complete desktop application in Python using the Tkinter GUI toolkit.
- Ability to design a normalized relational database in MySQL and connect it to a Python application using mysql-connector-python.
- Practical understanding of the MVC (Model-View-Controller) architecture and its benefits in real projects.
- Proficiency in implementing complete CRUD operations, JOIN-based reporting queries, and secure login with password hashing.
- Hands-on skill in exporting reports to PDF, Excel, and CSV formats using reportlab, openpyxl, and the csv module.
- Practical understanding of the Software Development Life Cycle (SDLC) — requirement analysis, design, coding, testing, and documentation.
- Competence in using professional tools: Git/GitHub, VS Code/PyCharm, MySQL Workbench, and Python's unittest and logging modules.
- Improved soft skills including teamwork, communication, time management, documentation, and presentation skills.
- Clear understanding of workplace culture, professional ethics, and the expectations of the IT industry, which has significantly enhanced my employability.

6. CONCLUSION / SUMMARY

This project-based training course at ZillionSoftech, Rewari, proved to be an extremely valuable learning experience. It gave me the opportunity to step out of theoretical learning and experience the actual working environment of the software industry.

During the course, I successfully developed a complete Hostel Management System using Python, Tkinter, and MySQL, following the professional MVC architecture. I designed a normalized database, implemented secure login, full CRUD operations for residents, rooms, wardens, and fees, dynamic pending-fee calculation, complaint tracking, and multi-format report generation. The application was unit tested, manually tested, documented, and demonstrated successfully.

The experience strengthened my technical foundation in Python programming and database management, sharpened my problem-solving and debugging skills, and developed professional qualities such as discipline, teamwork, and effective communication. I also understood how a real project moves through the phases of the Software Development Life Cycle, from requirement analysis to final delivery.

This course has given me clarity about my career direction and the confidence to take up job roles in software development and database management. I strongly believe that the knowledge and experience gained during this training will serve as a strong foundation for my professional career.

7. ATTACHMENTS

The following documents are attached with this report:

- Course/Training Completion Certificate issued by ZillionSoftech, Rewari.
- Additional screenshots of the developed application.
- SQL script of the database (hostel_db.sql).
- Sample exported reports (PDF/Excel) generated by the application.
- Weekly attendance / work diary duly signed by the Course Trainer/Mentor.

[Attach the above documents after this page]